Comparing Classical and Deep Learning Models on NASA's SMAP Satellite Telemetry for Anomaly Detection

Katie Craft
Sproul.4@wright.edu

Abstract**—**Satellites are an incredibly expensive asset to researchers and government agencies, and any issues with these systems can cause a detrimental mission failure. Early warning signs such as anomaly detection is essential for lengthening the satellites longevity. This study addresses the use of machine learning to detect contextual anomalies in satellite telemetry data such as the NASA SMAP dataset by comparing three models, a classical isolation forest, a deep learning variational autoencoder (VAE), and a deep learning Long Short-Term Memory VAE (LSTM-VAE). Classic models tend to underperform compared to the deep learning models due to their lack of learned patterns. This study found that the VAE obtained the highest F1-score at 0.37, whereas the LSTM-VAE had an F1-score of 0.34, followed by the isolation forest which attained an F1-score of 0.087. This shows the deep learning models greatly improved the models' ability to learn the long term patterns within the data. This comparison provides a stepping stone to the popularity of utilizing machine learning models for real-world satellite operations.

Keywords—machine learning, classical, isolation forest, deep learning, variational autoencoder (VAE), long short-term memory (LSTM), sliding window, NASA SMAP/MSL, time series

GitHub Link: https://github.com/kcraft10/kc-satellite

I. Introduction

Anomaly detection in satellite telemetry data is extremely important for the overall health of these expensive machines. Many agencies expend time, effort, and money to get these assets working and deployed into our atmosphere. Detecting when these satellites have system failures is crucial for being able to identify the malfunction and rush to fix it before further damage can occur. The role of machine learning can automate and expedite this process, saving companies and government agencies time and money, while also relieving the stress of satellite operators. These operators need to be experts in the field, to be able to understand the sensors and notice when there is something wrong. Adding in machine learning models that can detect these anomalies and potentially be able to alert operators before the entire system crashes is the next step in this technologically fast-paced world.

This topic has been experimented with previously, using both classical machine learning techniques as well as deep learning techniques to detect anomalies. Studies have found that the deep learning models tend to perform better than the simple models because of their improved ability to understand the patterns present in the telemetry data [1].

This study compares three models: an isolation forest model, a variational autoencoder (VAE) model with the use of a sliding window, and an LSTM-VAE model on NASA's Soil Moisture Active Passive (SMAP) P-1 channel dataset. This comparison aids in learning how these models can understand contextual telemetry data, and whether or not deep learning models outperform traditional machine learning models. In this work:

- Isolation Forest is compared to a Variational Autoencoder (VAE) and an LSTM-VAE in detecting anomalies in NASA's SMAP P-1 channel.
- An increase in precision, recall, and F1-score is obtained in total between the classic and deep learning models

II. Related Work

Classical isolation forest has been used to detect anomalies previously [2]. Deep learning models have been popularized and used regularly including Recurrent Neural Networks (RNN), Convolutional Neural Networks (CNN), Transformers, Long Short-Term Memory (LSTM) models and many others [3]. J. Rewicki [1] used an autoencoder (AE) model on a University of California, Riverside (UCR) dataset containing univariate datasets to achieve an F1-score of 0.16 with a sliding window of 10 time steps. This current study introduces the use of a longer sliding window (50 time steps) in the VAE deep learning model and an addition of an LSTM-VAE to allow the model to compare the current data point with previous data points to gain insight in the normal values as well as introduce memory. This comparison illustrates the added benefits of using a deep learning model with a sliding window for anomaly detection.

III. Methodology

A. Model Architecture

The models proposed here are a classical isolation forest model, a deep learning variational autoencoder (VAE) model, and a deep learning long short-term memory (LSTM) VAE to detect anomalies in time series telemetry data.

Classical Model (Isolation Forest):

This model was implemented using 1000 estimators due to the high dimensionality. Various values were experimented with for the number of trees in the ensemble, but this value gave the most optimal performance to understand the complexity of the data without costing a large amount of efficiency.

The contamination value defines the proportion of outliers in the dataset, and was experimented with in this analysis [4]. It was ultimately chosen to be 0.08. This is due to the rare nature of these anomalies in the telemetry data. Anomalies are so rare in satellite telemetry, that a lower value of contamination was chosen to be able to limit the false positive rate.

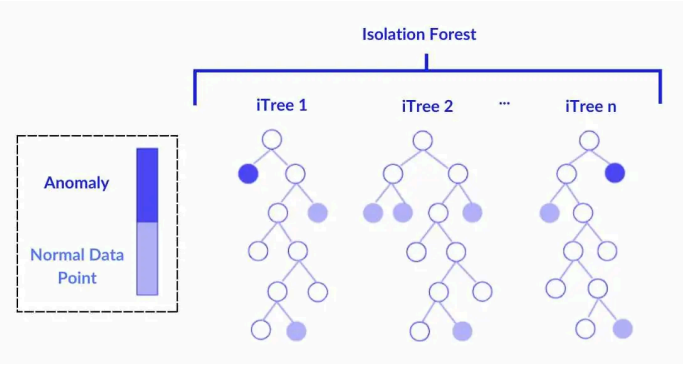

Fig 1. Architecture of the isolation forest mechanism.

Deep Learning Model (Variational Autoencoder (VAE)):

Sliding Window:
The deep learning VAE model was implemented first by using a sliding window technique. This technique allows for the model to learn more contextual data as it takes a sequence of data rather than a single row of data [5]. The window size was set to 50 time steps, which attempted to provide enough context for future observations, while not too large to introduce noise. The model can then take each row and compare it to the previous rows, allowing it to see more of the contextual story [5].

VAE:
The VAE model was implemented using two latent dimensions. This is the bottleneck layer, which is a compressed version of the input [6]. The latent dimension layer was experimented with but a large change

in performance was not noticed when increasing this parameter. Similarly, the encoder and decoder architecture was experimented with, but the following architecture minimized the error. The encoder and decoder were both created using one hidden layer with 8 nodes. The activation function used was the sigmoid function, which gave the most optimal performance to minimize false negatives. The threshold for anomalies was 0.88, meaning the top 0.12 of points were classified as anomalies for this model. This threshold was varied in experimentation, but this value provided the most true positives without sacrificing many false negatives.

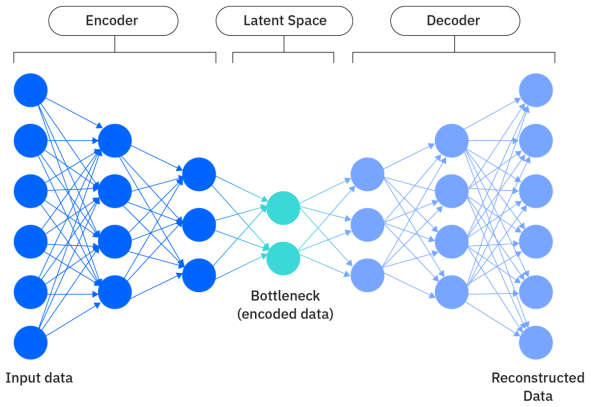

Fig. 2. Architecture of the VAE.

Deep Learning Model (LSTM-VAE):

The hybrid model was implemented to see if this addition of an architecture that can preserve the contextual data would improve the performance. This technique allows the model to take information from previous inputs and use it for the following output [7]. This model was implemented using the same sliding window as mentioned in the VAE model. This utilized a window size of 50. This was kept consistent with the VAE for simplicity. The LSTM-VAE was implemented using 16 latent dimensions. The shape of the data was also preserved to be able to show the underlying patterns. The encoder utilized the one LSTM layer with 8 nodes and the sigmoid activation function. These hyperparameters were experimented with, but this layout gave the most optimal performance. A dropout rate of 0.10 was used in the encoder as well to prevent overfitting. The decoder also used one layer with 8 nodes and the sigmoid function. A dropout rate of 0.10 was also used. The output activation function was sigmoid to determine if the points were anomalous or normal. The VAE was then implemented similarly to the previous model, but this time using 30 epochs, with a batch size of 1000. The loss used was MSE due to the goal being wanting to recreate the input, therefore we would need to measure the distance between the input and recreation of the input. The threshold of 0.85 was used, where the top 0.15 proportion of data was able to be classified as anomalous. Again, this was experimented with, but ultimately was the most optimal to balance the false negative rate and false positive rate.

The structure of the LSTM-VAE model follows similar to Fig 2, whereas the layers consist of LSTMs rather than dense layers. These LSTMs are recurrent neural networks (RNNs) that preserve memory of the data by feeding back the input into itself then deciding what is important or not to keep.

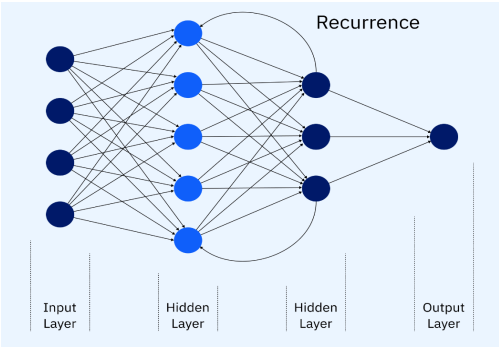

Fig 3. Architecture of the LSTM process within the LSTM-VAE.

B. Training Procedure

It's important to note that while the expert-labeled anomalies were given, this feature was not implemented into the training to avoid data leakage during training.

The isolation forest training was done by fitting the training data then predicting the test data by generating an anomaly score for each point (-1 for anomaly, 1 for normal). These values were converted to 0 and 1 to be used in the confusion matrix analysis.

The VAE training was done similar to the isolation forest mechanism but using the training window of 50 time steps, with 20 epochs and a batch size of 500, which minimized the error. The VAE was trained on the training data then predicted the telemetry values as a reconstructed sequence, then calculated the loss (MSE) between the reconstructed points and the actual values. The threshold was then set to divide the telemetry into anomalous and normal points for classification and confusion matrix analysis.

The LSTM-VAE training was done similar to the VAE, using a sliding window of 50. The LSTM-VAE was trained on the training data then recreated the telemetry values, followed by the loss calculation (MSE) to determine the distance between the input and the reconstructed input. The threshold was then set to classify the points as normal or anomalous for the confusion matrix and performance metrics steps.

IV. Experimental Setup

A. NASA SMAP Dataset

The "NASA Anomaly Detection Dataset SMAP and MSL" consisted of the P-1 channel dataset that was previously split by the authors, containing a train and test dataset [8]. A summary of the dataset used is below:

TABLE I. ***Summary of SMAP Dataset***

Dataset	Number of Observations	Split (%)	Anomalous Events (With Index Sequence)
P-1 Train	2872	~25%	0
P-1 Test	8505	~75%	3 ([2149, 2349], [3539, 3779], [4536, 4844])

This split by the author gave the data approximately a 75% train and 25% test. This split was used because the model needs less data to learn the normal satellite behavior, whereas it needs more data to see the rare anomalies in the test data. Each dataset consisted of 25 columns, the first being the normalized telemetry values from -1 to 1. The other 24 columns consisted of one hot encoded values that depended on whether commands were given to the satellite or not. If there is a "1" in a certain column, this indicates a command was given to the satellite, and if there are all "0's" in that row, no command was given.

A ground truth label dataset was also provided in a separate csv file. This contained the sequence of values that were expert-labeled anomalies for the P-1 channel. The anomalies consisted of three separate events with the timestamp sequences [2149, 2349], [3539, 3779], and [4536, 4844]. These labels allowed us to check the models' performance.

The dataset considered has a unique structure as the first feature is the normalized telemetry (sensor) value, followed by 24 one-hot encoded features. These elements are one-hot encoded for whether or not a command is given to the satellite or not, such as adjusting instruments, data transmission, or altitude adjustment [9].

TABLE II. ***Sample of SMAP P-1 Channel Dataset***

Index/ Feature	0 (Telemetry)	1	…	24
0	-0.52	0	0	1
1	-0.69	0	0	0
…	-0.55	1	0	0
2872	-0.53	0	0	0

Since the data was already normalized, the pre-processing was minimal for the analysis. The pre-processing done included adding time step indices and creating a label column to place the binary value for normal/anomalous events.

The training dataset is graphed below:

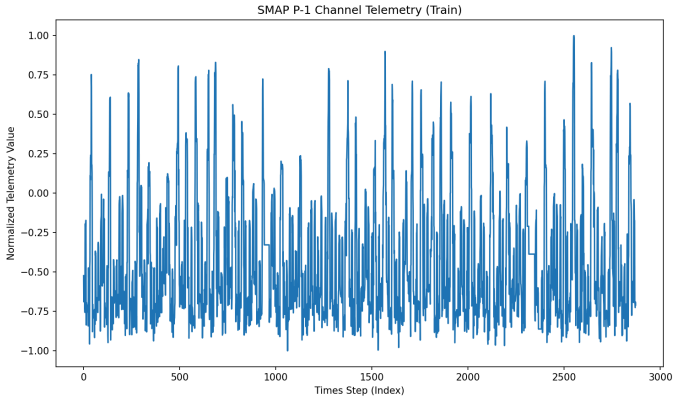

Fig 4. Telemetry of the P-1 channel (training dataset).

The graph of the testing dataset is below, with expert-labeled anomaly sequences shown:

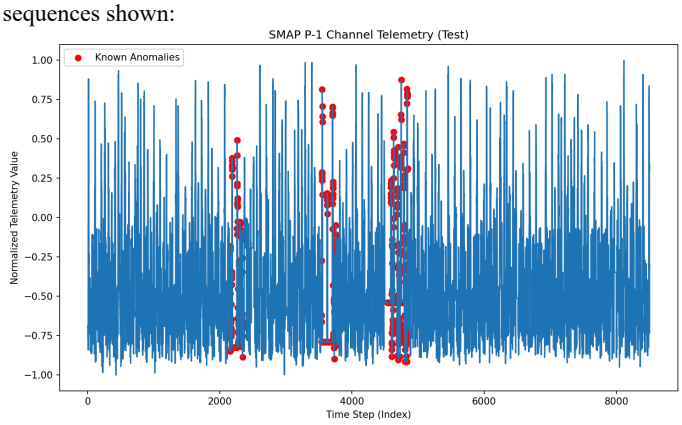

Fig 5. Telemetry of the P-1 channel (testing dataset) with labeled anomalies overlaid.

B. Evaluation Metrics

The metrics used in this study included precision, recall, and F1-score. These metrics were used to obtain the full picture of the models' performance by presenting the correctness, and how well the models were able to identify the anomalies as these are the most important for this task. It is extremely important to understand the true positives in terms of all positives predicted (precision) and the true positives in terms of actual positives (recall). It is also important to view the harmonic mean between these two metrics (F1-score) when there is class imbalance. Accuracy was not reported as the high class imbalance in the data would deceptively inflate the accuracy for the models. Prioritizing the recall is the best choice here as a large number of true positives is important as well as a low number of false negatives, so the machine learning model does not miss anomalies is crucial. This metric was attempted to be maximized here.

The confusion matrices were also presented to compare the overall predictions between the two models. These values were also used in a Precision-Recall curve to view the trade-off between these two metrics for each model.

C. Implementation Details

The isolation forest model used a random state of 20 and a contamination rate of 0.08 for reproducibility. The VAE model utilized a sliding window length of 50 time steps, 20 epochs, and a batch size of 500 for implementation. The LSTM-VAE used a sliding window length of 50 time steps, 30 epochs, and a batch size of 1000 for implementation.

V. Results

A. Main Results

Isolation Forest:

Hyperparameter Tuning:
Various hyperparameters were experimented with during the training process, and the resulting performance metrics are below. Specifically, the number of estimators (number of base trees) and contamination rate (proportion of outliers) was varied and compared to obtain the most optimal training conditions.

TABLE III. ***Summary of Performance Metrics for N_estimators***

N_estimators	Precision	Recall	F1-Score
50	0.082	0.099	0.090
100	0.084	0.10	0.091
1000	**0.085**	**0.10**	**0.092**
10000	0.086	0.10	0.093

As the number of estimators (number of base trees) increased, the performance increased marginally. Based on the metric, a number of estimators of 1000 was chosen to balance performance and efficiency.

TABLE IV. ***Summary of Performance Metrics for Contamination Rate***

Contamination	Precision	Recall	F1-Score
0.0001	0.0	0.0	0.0
0.001	0.0	0.0	0.0
0.01	0.084	0.012	0.021
0.02	0.072	0.016	0.026
0.05	0.091	0.056	0.069
0.08	**0.088**	**0.085**	**0.087**
0.1	0.085	0.10	0.092

As the contamination rate increased, the number of false positives increased substantially, but the false negatives decreased. The recall continued to increase, but since the false positives continued to increase, the implications of a real-world scenario outweighed the goal of obtaining the highest recall performance. If the contamination rate is too large, the model will flag too many points to where it becomes useless. Based on these results, a contamination value of 0.08 was chosen to balance the performance and false positive rate.

Performance:
For the isolation forest model, the telemetry was graphed with the model's predicted anomalies shown.

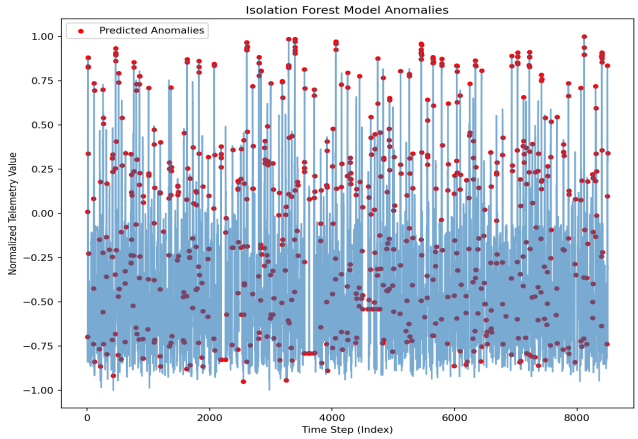

Fig 6. Isolation forest model's predictions shown on the telemetry data.

The resulting confusion matrix is below:

TABLE V. ***Isolation Forest Confusion Matrix***

Isolation Forest	Predicted 0	Predicted 1
Actual 0	7092	662
Actual 1	687	64

The model was able to obtain 64 true positives, 687 false negatives, 662 false positives, and 7092 true negatives. The large number of true negatives is due to the heavy class imbalance. There are a large number of both false positives and false negatives which is undesirable in the context of this problem.

VAE:

Hyperparameter Tuning:
The hyperparameters were experimented with during the training process and the resulting performance metrics are below. The hyperparameters that were tuned include the sliding window size, number of latent dimensions (compressed bottleneck layer size), number of hidden layers, number of nodes, activation functions, epochs, batch size, choice of loss used, and loss threshold for prediction to obtain the most optimal training conditions.

TABLE VI. ***Summary of Performance Metrics for Sliding Window***

Sliding Window Size	Precision	Recall	F1-Score
10	0.11	0.15	0.13
50	**0.20**	**0.28**	**0.23**
100	0.19	0.25	0.22
150	0.15	0.20	0.17
200	0.12	0.16	0.13
250	0.053	0.072	0.061
500	0.13	0.182	0.15

As the sliding window size increased, the metrics seemed to peak then decline. There was a peak performance at a window size of 50, then it declined over the next window sizes. The false negatives and false positives continued to increase as the sliding window increased. There was a steep decline in performance between the window size of 200 and 250. This could have been because the relative size of the anomalous events from the test set was approximately the length of 200-300 points. Ultimately the optimal choice seemed to be a window size of 50.

TABLE VII. ***Summary of Performance Metrics for Latent Dimensions***

Latent Dimension Size	Precision	Recall	F1-Score
1	0.20	0.28	0.23
2	**0.20**	**0.28**	**0.24**
5	0.20	0.28	0.23
10	0.20	0.28	0.24
20	0.20	0.28	0.24
50	0.20	0.28	0.24

For the various values of latent dimensions, this seemed to have little effect on the performance of the model. This could be due to the fact that the meaningful data was already captured by the smaller number of dimensions. The chosen latent dimensions were 2 for simplicity.

TABLE VIII. ***Summary of Performance Metrics for Layers/Nodes***

Layers/Nodes	Precision	Recall	F1-Score
1/4	0.23	0.31	0.26
1/8	**0.21**	**0.29**	**0.24**
1/16	0.20	0.28	0.23
1/32	0.20	0.27	0.23
2/4,2	0.23	0.31	0.26
2/8,4	0.21	0.29	0.24
2/16,8	0.20	0.28	0.24
2/32,16	0.20	0.27	0.23

One hidden layer with 8 nodes was chosen to be able to keep both the encoder and decoder symmetric as well as the reconstruction loss high as the normal patterns in the data is what we want to create. Then if it sees an anomaly it can recognize it quickly. This architecture also keeps the model as simple as possible.

The activation function was tested to see which gave the best performance. The ReLU and Sigmoid were tested as combinations as this problem is binary classification, but wanted to see if the vanishing gradient was a problem. Tanh was not tested here as the output needed to be non-negative.

TABLE IX. ***Summary of Performance Metrics for Activation Functions***

Activation Function (Encoder/Decoder)	Precision	Recall	F1-Score
ReLU/ReLU	0.19	0.26	0.22
Sigmoid/Sigmoid	**0.21**	**0.28**	**0.24**
ReLU/Sigmoid	0.20	0.27	0.23

The Sigmoid activation function for both the encoder and decoder layer was the most optimal, possibly due to the fact that the output range was between 0 and 1 to be able to classify the points as anomalous, where the ReLU function's range is much larger, inhibiting the model's classification abilities. Ultimately the Sigmoid function was chosen as the encoder/decoder activation function as well as the output function. Using this function allows it to be able to have the non-linearity present, to learn the relationships in the data and how the telemetry fluctuates.

TABLE X. ***Summary of Performance Metrics for Epochs***

Epochs	Precision	Recall	F1-Score
20	**0.22**	**0.30**	**0.25**
30	0.21	0.29	0.24
50	0.20	0.28	0.23
100	0.20	0.27	0.23
500	0.18	0.24	0.21
1000	0.22	0.30	0.25

The choice of 20 epochs was chosen to minimize the loss, indicating the model is learning the data and reconstructing it well.

TABLE XI. ***Summary of Performance Metrics for Batch Size***

Batch Size	Precision	Recall	F1-Score
50	0.20	0.28	0.24
100	0.21	0.29	0.24
150	0.22	0.30	0.25
250	0.28	0.37	0.32
500	**0.31**	**0.43**	**0.36**
1000	0.27	0.37	0.31

The recall and precision seemed to increase as the batch size increased, peaking at a batch size of 500. This is because there was a decrease in the false negatives. The optimal batch size was 500, which is likely because this allowed the model to get a good understanding of what the normal values look like, then can differentiate more between the anomalous points and the normal points.

Two types of loss choices were also compared.

TABLE XII. ***Summary of Performance Metrics for Loss***

Loss	Precision	Recall	F1-Score
BCE	0.063	0.085	0.072
MSE	**0.32**	**0.43**	**0.37**

The MSE provided much larger metrics likely because the data contains both continuous values and discrete, therefore the MSE just calculates the distance between the output and input. The BCE penalizes the continuous values and provides much lower metrics. The MSE was ultimately chosen as the type of loss used.

TABLE XIII. ***Summary of Performance Metrics for Anomaly Threshold***

Anomaly Threshold	Precision	Recall	F1-Score
0.80	0.21	0.48	0.29
0.85	0.26	0.45	0.33
0.88	**0.32**	**0.43**	**0.36**
0.90	0.36	0.40	0.38
0.95	0.63	0.36	0.46
0.99	1.0	0.11	0.21

As the threshold of anomaly classification increases, there were fewer points detected as anomalies as the model could only classify a certain number of them as anomalies. The optimal threshold was 0.88 as this kept the number of false negatives and false positives the lowest values, while also being able to still classify points as anomalies.

Performance:
For the VAE, the model was run and the model's predictions of anomalies were graphed over the telemetry data.

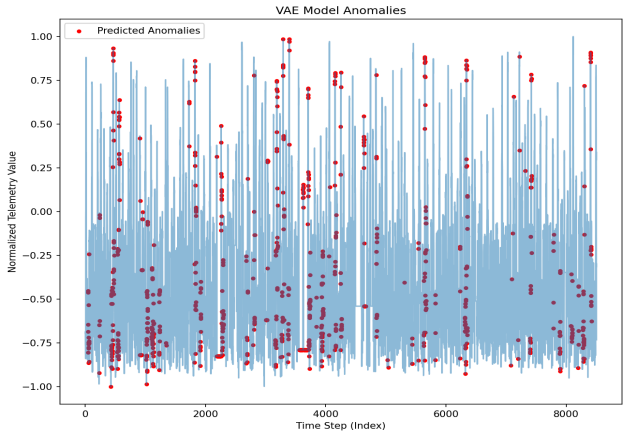

Fig 7. VAE model's predictions shown on the telemetry data.

The resulting confusion matrix is below:

TABLE XIV. ***VAE Confusion Matrix***

VAE	Predicted 0	Predicted 1
Actual 0	7054	700
Actual 1	430	321

The VAE found 321 true positives, 700 false positives, 430 false negatives, and 7054 true negatives. Similarly, there are a large

number of false positives and false negatives, which is undesirable for this context.

LSTM-VAE:

Hyperparameter Tuning:
Various hyperparameters were experimented with during the training process, and the resulting performance metrics are below. More specifically, the latent dimensions, number of layers, number of nodes, activation functions, dropout rate, epochs, batch size, and loss threshold was varied and compared to obtain the most optimal training conditions.

TABLE XV. ***Summary of Performance Metrics for Latent Dimensions***

Latent Dimension Size	Precision	Recall	F1-Score
1	0.35	0.20	0.26
2	0.33	0.19	0.24
5	0.36	0.20	0.26
8	0.31	0.18	0.23
10	0.35	0.20	0.25
16	**0.35**	**0.20**	**0.26**
20	0.35	0.20	0.25

For the various values of latent dimensions, the performance did not change much between the size of the bottleneck. The data seemed captured by the various numbers of latent dimensions, therefore, the chosen value was 16.

TABLE XVI. ***Summary of Performance Metrics for Layers/Nodes***

Layers/Nodes	Precision	Recall	F1-Score
1/8	**0.64**	**0.36**	**0.46**
1/16	0.49	0.28	0.35
1/32	0.27	0.15	0.19
1/64	0.26	0.15	0.19
2/16, 8	0.44	0.25	0.32
2/32, 16	0.36	0.21	0.26
2/64, 32	0.24	0.14	0.18

One hidden layer with 8 nodes was chosen to be able to keep both the recall as large as possible to minimize false negatives. This also kept the encoder and decoder symmetric and the reconstruction loss high.

The activation functions were tested to see which combination gave the best performance.

TABLE XVII. ***Summary of Performance Metrics for Activation Functions***

Activation Function (Encoder/Decoder)	Precision	Recall	F1-Score
ReLU/ReLU	0.54	0.31	0.39
Sigmoid/Sigmoid	**0.64**	**0.36**	**0.46**
ReLU/Sigmoid	0.62	0.35	0.45

The Sigmoid activation function for both the encoder and decoder layer was the most optimal, likely due to the binary output range. The Sigmoid function was used as the encoder/decoder function and the output function.

TABLE XVIII. ***Summary of Performance Metrics for Dropout***

Dropout Rate	Precision	Recall	F1-Score
0.08	0.63	0.36	0.46
0.10	**0.64**	**0.36**	**0.47**
0.20	0.64	0.36	0.46
0.30	0.64	0.36	0.46
0.40	0.63	0.36	0.45
0.50	0.63	0.36	0.46

For the various proportion of dropout rate, this seemed to have little impact on the performance. A smaller value was chosen to be able to decrease underfitting and allow the model to attempt to fully learn the patterns. The value chosen was 0.10 for the dropout rate.

TABLE XIX. ***Summary of Performance Metrics for Epochs***

Epochs	Precision	Recall	F1-Score
20	0.64	0.36	0.47
30	**0.66**	**0.37**	**0.48**
50	0.38	0.21	0.27
100	0.28	0.16	0.21

The choice of 30 epochs was chosen to minimize the false negatives and maximize the true positives. The increase in epochs seemed to make the model perform worse, indicating the model took fewer epochs to fully learn as much information as possible.

TABLE XX. ***Summary of Performance Metrics for Batch Size***

Batch Size	Precision	Recall	F1-Score
50	0.24	0.14	0.18
100	0.24	0.14	0.18
150	0.27	0.16	0.20
250	0.37	0.21	0.27
500	0.66	0.37	0.48
1000	**0.27**	**0.45**	**0.34**

The recall and precision seemed to increase as the batch size increased, peaking at a batch size of 500. On the other hand, the confusion matrix was the most optimal case for a batch size of 1000, as this was able to minimize the false positive rate. Therefore the batch size chosen was 1000.

TABLE XXI. ***Summary of Performance Metrics for Anomaly Threshold***

Anomaly Threshold	Precision	Recall	F1-Score
0.80	0.21	0.48	0.30
0.82	0.23	0.47	0.31
0.85	**0.27**	**0.46**	**0.34**
0.88	0.26	0.35	0.29
0.90	0.34	0.38	0.36
0.95	0.65	0.37	0.47
0.99	0.87	0.10	0.18

The best case to have the fewest false negatives while also detecting true positives was 0.85. This allowed the model to decrease the false negatives while still detecting true positives.

Performance:
For the LSTM-VAE, the model was run and the model's predictions of anomalies were graphed over the telemetry data.

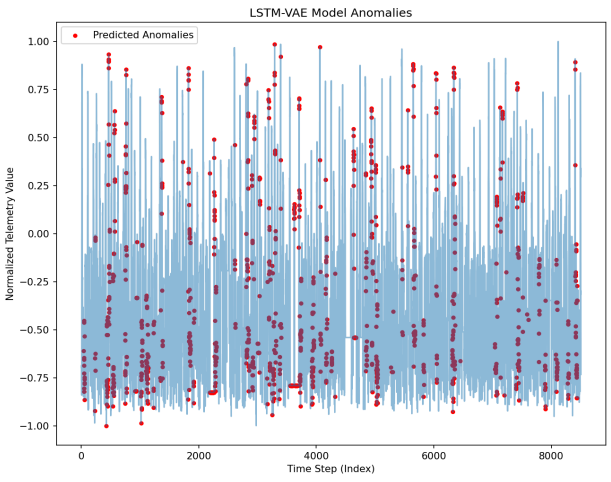

Fig 8. LSTM-VAE model's predictions shown on the telemetry data.

The resulting confusion matrix is below:

TABLE XXII. LSTM-*VAE Confusion Matrix*

LSTM-VAE	Predicted 0	Predicted 1
Actual 0	6819	935
Actual 1	410	341

The LSTM-VAE found 341 true positives, 935 false positives, 410 false negatives, and 6819 true negatives. Similar to the VAE, there are a large number of false positives and false negatives, which is undesirable for this context, but there was a decrease in false negatives, which is an improvement. This shows that the model was able to be more sensitive to the anomalies present, increasing the recall. While there was a large number of false positives, these hyperparameters gave the best balance between these values as well as increasing the number of true positives.

Isolation Forest versus VAE versus LSTM-VAE Performance:

To look into the models' performance, the Precision-Recall Curve was generated.

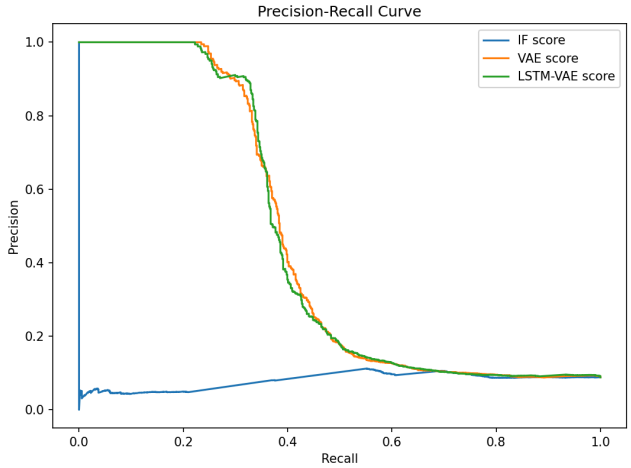

Fig 9. Precision-Recall curve of the three models: Isolation Forest, VAE, and LSTM-VAE.

The most optimal curve would be close to the top right corner, therefore the isolation forest model is showing that it cannot differentiate between the two classes of anomalous versus normal. The VAE model is closer to the top right corner, but still unable to balance the precision and recall together. Similarly, the LSTM-VAE follows pretty close to the VAE line, as the two models cross each other. This indicates they both can better differentiate between anomalies and normal points compared to the isolation forest, but cannot fully learn the patterns.

The three models were also compared by their calculated performance metrics:

TABLE XXIII. ***Summary of Performance Metrics***

Model	Precision	Recall	F1-Score
Isolation Forest	0.088	0.085	0.087
VAE	**0.32**	**0.43**	**0.37**
LSTM-VAE	0.27	0.45	0.34

The LSTM-VAE model had a lower precision, higher recall, and lower F1-score compared to the VAE model, but higher metrics compared to the Isolation Forest model.

Based on these results, the chosen model would be the VAE, as this is the simpler, more explainable model compared to the LSTM model. While the recall was slightly higher for the LSTM model, it is not enough of an increase to really say it added anything to the model's abilities. Considering the confusion matrix was able to best balance false negatives and false positives, the simpler of the two deep learning models would be the best choice here.

VI. Discussion

The isolation forest model seemed to predict anomalous points scattered across various telemetry values, with a lot of anomalies near the peaks of the telemetry data. This was evident in the confusion matrix as there were not many true positive predictions. This indicates that the simple isolation forest model could not learn the patterns that were present in the data due to the data's dependent structure. This is because the fluctuations in the data does not only depend on the general spikes and dips of the telemetry, but depends on the one-hot encoded features as well. The model would need to learn the pattern of the given commands and the relative telemetry value to be able to discern between anomalies or normal data points.

The simple model also found a large number of true negatives at 7092. This was likely due to the heavy class imbalance, making the classification seem better than it is. The false positive rate and false negative rates were similar and high. This is detrimental to the domain of satellite operations. If there are constant flags for anomalies when there are none, this essentially renders the model useless. Likewise, if there are many anomalies where the model misses that they are anomalous points, the satellite will fail. The isolation forest had a precision of 0.088, a recall of 0.085, and an F1-score of 0.087. This is a very poor performance as all metrics would not be very useful at determining anomalies present in the data. This model is essentially guessing rather than learning. It is not built for contextual anomalies because it does not preserve the temporal patterns that a deep learning model could.

The deep learning VAE model had a slight increase in performance. While the VAE had more false positives, it had fewer false negatives which is an improvement in this case. The model was able to predict 321 true positives which is also an improvement from the isolation forest model. The VAE had a precision of 0.32, a recall of 0.43, and an F1-score of 0.37. Overall, this is a fairly poor performance, but relative to the simple model, it is an improvement. This shows the deep learning model was able to learn some of the patterns present in the data, but still was not able to fully learn and interpret the contextual shifts based on the telemetry and one-hot encoded features together.

The LSTM-VAE model seemed to detect larger clumps compared to the previous models and shows detecting slices of the telemetry rather than scattered points. This looks to be a better indication of detecting the anomalies based on the plot alone. The LSTM-VAE model was able to detect more true positives and fewer false negatives than the VAE model. This was an improvement, but it also detected more false positives, which is not great. This would be harmful to the satellite missions as continuous alerts render the model useless. This model had a lower precision, higher recall and lower F1-score which was an average performance compared to the VAE. This additional LSTM layer did not seem to have as large of an impact as expected. This could be due to the fact that the data, while normalized and clean without missing data, could be too noisy and unpredictable for these models. This model had a precision of 0.27, a recall of 0.45, and an F1-score of 0.34. This is a fairly poor performance still, but comparable to the VAE model. It shows an improvement from the simple isolation forest, as well as some indication of learning more of the patterns present in the data.

The Precision-Recall curve showing the tradeoff between the three metrics shows that the isolation forest model has a very hard time differentiating between normal and anomalous points and cannot obtain either a reliable precision nor recall. The VAE shows that it can differentiate better than the isolation forest model but is still under the halfway point between having a high precision and recall. The LSTM-VAE showed a similar pattern as the two deep learning models overlapped in the plot. This means the shift between a high recall and high precision is very abrupt and it cannot differentiate well between the classes either because it misses many of the positive cases of anomalies. The models experience a large cut off in performance around the recall of around 0.40. This means the model cannot differentiate even half of the anomalies present.

There could be a few reasons why the models did not perform well. One potential reason would be that time series data is generally noisy and this could be influencing the learning the models are trying to. Another reason is because of the types of anomalies in this dataset. The anomalies are contextual where they need additional input to be able to make a decision whereas something like point anomalies, which are anomalies that are outside of a specific value range. This added knowledge within the data is extremely important to the classification of the anomalies, and if the model cannot fully grasp the extent of the knowledge, it will underperform. As for the specific models, the isolation forest could be too simple of a model to really understand the temporal context. As for the VAE, while it did improve, it also could not have fully understood the contextual anomalies well enough to detect the anomalies.

Overall, the isolation forest model did not perform well for this type of data. The VAE showed an improvement compared to the isolation forest, but was not fully able to learn and understand the complex data structure and dependency within the dataset. Similarly, the LSTM-VAE showed an improvement compared to the isolation forest model, but performed similar to the VAE. These models could be better used on point anomalies where the outliers are outside of a certain range of values to be able to be detected by these machine learning models.

Limitations and Future Work:

This particular dataset is incredibly clean, as it is normalized and ground truth labels were provided. A robust and generalizable model would need to learn complex patterns and be able to be trusted enough for operators to use it. The sliding window can be considered computationally heavy, and therefore not as useful when implementing this architecture on actual satellites. This leads to the future work that could be done such as testing these models with different datasets that contain different anomalies. Experimenting with other sliding window lengths or even less computationally rich sliding windows could be another aspect worth analyzing. Introducing aspects of explainability in the different sensors to determine the causes for anomalies is another potential route that could be explored.

VII. Conclusion

In this paper, an isolation forest model, a VAE, and an LSTM-VAE were compared to achieve an increase in precision, recall, and F1-score for the deep learning models. Specifically, the isolation forest model capped at a precision of 0.088, a recall of 0.085, and F1-score of 0.087 because it was unable to learn the complex patterns present in the data. The VAE model attained a precision of 0.32, a recall of 0.43, and F1-score of 0.37 due to the learned patterns from the more complex model, but still struggled with false positives. The LSTM-VAE model attained a precision of 0.27, a recall of 0.45, and an F1-score of 0.34, as it learned some of the complex patterns present. Future directions include testing these models with real-world datasets and experimenting with other deep learning models on this dataset.

References

[1] F. Rewicki, J. Denzler, J. Niebling, "Is It Worth It? Comparing Six Deep and Classical Methods For Unsupervised Anomaly Detection in Time Series," 2023, arXiv:2212.11080. [Online]. Available: https://arxiv.org/abs/2212.11080.

[2] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation Forest," IEEE Int. Conf. Data Min., 2008, pp. 413-422.

[3] Z. Z. Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, "Deep Learning for Time Series Anomaly Detection: A Survey,"

ACM Comput. Surv., vol. 57, no. 1, article 15, pp. 1-42, Oct. 2024.
[4] "Isolation Forest." Scikit-learn. 14 Apr. 2026 [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html.
[5] "What is a sliding window approach in time series forecasting?" Milvus. 10 Apr. 2026 [Online]. Available: https://milvus.io/ai-quick-reference/what-is-a-sliding-window-approach-in-time-series-forecasting.
[6] N. Somani. "Variational Autoencoders (VAEs): Understand the Working." igmGuru. 6 Apr. 2026 [Online]. Available: https://www.igmguru.com/blog/variational-autoencoders.
[7] J. Noble, "Long short-term memory," IBM.com, 24 Apr. 2026 [Online]. Available: https://www.ibm.com/think/topics/lstm.
[8] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, T. Soderstrom, "Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding," KDD, 2018, pp. 387-395.
[9] F. Quiquet, "Description of the Elements of a Satellite Command and Control System," Space & Cybersecurity. 24 Apr. 2026 [Online]. Available: https://www.spacesecurity.info/en/description-of-the-elements-of-a-satellite-command-and-control-system/.
[10] "Anomaly detection using Isolation Forest," GeeksforGeeks, 6 Apr. 2026 [Online]. Available: https://www.geeksforgeeks.org/machine-learning/anomaly-detection-using-isolation-forest/.
[11] "Isolation Forest example," Scikit-learn. 6 Apr. 2026 [Online]. Available: https://scikit-learn.org/stable/auto_examples/ensemble/plot_isolation_forest.html#sphx-glr-auto-examples-ensemble-plot-isolation-forest-py.
[12] "pandas.DataFrame.index," Pandas. 6 Apr. 2026 [Online]. Available: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.index.html.
[13] "matplotlib.axes.Axes.scatter," Matplotlib. 6 Apr 2026 [Online]. Available: https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.scatter.html.
[14] piyushnaik, "Sliding Window with Isolation forest," Kaggle. 6 Apr. 2026 [Online]. Available: https://www.kaggle.com/code/piyushnaik/sliding-window-with-isolation-forest.
[15] "10 Common pitfalls and recommended practices," Scikit-learn. 6 Apr. 2026 [Online]. Available: https://scikit-learn.org/stable/common_pitfalls.html.
[16] ranasahira, "Isolation Forest and Rolling Mean for Handling Anomaly in Time Series Data," Medium. 6 Apr. 2026 [Online]. Available: https://medium.com/@sranasahira/isolation-forest-and-rolling-mean-for-handling-anomaly-in-time-series-data-5a0c6530234f.
[17] "NumPy linspace() Function," GeeksforGeeks. 6 Apr. 2026 [Online]. Available: https://www.geeksforgeeks.org/python/numpy-linspace/.
[18] A. Cook, "Data Leakage," Kaggle. 7 Apr. 2026 [Online]. Available: https://www.kaggle.com/code/alexisbcook/data-leakage.
[19] C. O'Sullivan, "Isolation Forest Guide: Explanation and Python Implementation," Datacamp. 7 Apr. 2026 [Online]. Available: https://www.datacamp.com/tutorial/isolation-forest.
[20] J. Guilherme, "How I Convert a string representation of a nested list into a nested list?," Stack Overflow. 7 Apr. 2026 [Online]. Available: https://stackoverflow.com/questions/71678738/how-i-convert-a-string-representation-of-a-nested-list-into-a-nested-list.
[21] "pandas.DataFrame.loc," Pandas. 7 Apr. 2026 [Online]. Available: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html.
[22] "What is Isolation Forest?," GeeksforGeeks. 14 Apr. 2026 [Online]. Available: https://www.geeksforgeeks.org/machine-learning/what-is-isolation-forest/.
[23] B. Dhupar, "Loss functions in Variational Autoencoders (VAEs)," Medium. 16 Apr. 2026 [Online]. Available: https://medium.com/@bhipanshudhupar/understanding-loss-functions-in-variational-autoencoders-vaes-a91aa1c10beb.
[24] "Practical determination of anomaly threshold in (variational) autoencoders," Stack Overflow. 17 Apr. 2026 [Online]. Available: https://stackoverflow.com/questions/50581043/practical-determination-of-anomaly-threshold-in-variational-autoencoders.
[25] "Methods to Minimize False Negatives and False Positives in Binary Classification," GeeksforGeeks. 12 Apr. 2026 [Online]. Available: https://www.geeksforgeeks.org/machine-learning/methods-to-minimize-false-negatives-and-false-positives-in-binary-classification/.
[26] "Variational AutoEncoders," GeeksforGeeks. 13 Apr. 2026 [Online]. Available: https://www.geeksforgeeks.org/machine-learning/variational-autoencoders/.
[27] V. Chugh, ""Precision-Recall Curve in Python Tutorial," Datacamp. 20 Apr. 2026 [Online]. Available: https://www.datacamp.com/tutorial/precision-recall-curve-tutorial.
[28] "Precision-recall curves - what are they and how are they used?" Acutecaretesting.org. 20 Apr. 2026 [Online]. Available: https://acutecaretesting.org/en/articles/precision-recall-curves-what-are-they-and-how-are-they-used.
[29] paya54, "Anomaly Detect LSTM VAE," GitHub. 20 Apr. 2026 [Online]. Available: https://github.com/paya54/Anomaly_Detect_LSTM_VAE/blob/master/lstm_vae.py.
[30] Aaron2Dev, "Variational Autoencoder on Time series with LSTM in Keras," Stack Overflow. 20 Apr. 2026 [Online]. Available: https://stackoverflow.com/questions/50874009/variational-autoencoder-on-timeseries-with-lstm-in-keras.
[31] N. V. Otten. "Isolation Forest For Anomaly Detection Made Easy & How To Tutorial." Spot Intelligence. 6 Apr. 2026 [Online]. Available: https://spotintelligence.com/2024/05/21/isolation-forest/.

Author Contributions

1st Author (Katie Craft): Conceptualization, Methodology, Experimentation, Formal analysis, Writing.
2nd Author (Dr. Tanvi Banerjee): Conceptualization, Supervision, Project Administration.